

VERİ TABANI YÖNETİMİ VE UYGULAMALARI DERSHANE ÖĞRENCİ TAKİP SİSTEMİ

Proje Raporu

Hazırlayanlar: F. Melek PINAR / Yasemin KURU

Numara: 240606007 / 240606010

Tarih: 21/12/2025

Adana Alparslan Türkeş Bilim ve Teknoloji
Üniversitesi – Bilişim Sistemleri ve Teknolojileri

1. GİRİŞ	3
1.1 Proje Amacı	3
1.2 Sistem Özeti	3
2. VERİTABANI TASARIMI	4
2.1 Varlık-İlişki Modeli	4
2.2 Tablo Yapıları	5
2.3 Anahtar (Key) Yapısı	5
3. UYGULAMA ÖZELLİKLERİ	8
3.1 Transaction Yönetimi	8
3.2 Trigger Kullanımı	10
3.3 İndeksleme Stratejisi	11
4. SORGULAR VE ANALİZLER	12
4.1 İstenen Sorgular	12
5. EKLER	14
EK A: Tüm SQL Kodları	14

1. GİRİŞ

1.1 Proje Amacı

Bu projenin amacı, bir dershanenin öğrenci takip işlemlerini verimli şekilde yönetmek için ilişkisel veri tabanı sistemi geliştirmektir. Sistem, öğrenci kayıtları, ödeme takibi, sınıf yönetimi ve öğretmen bilgilerini entegre şekilde yönetmeyi hedeflemektedir. Proje kapsamında veri bütünlüğünün sağlanması, iş kurallarının otomatik uygulanması ve performans optimizasyonu gibi temel veri tabanı yönetim prensipleri uygulanmıştır.

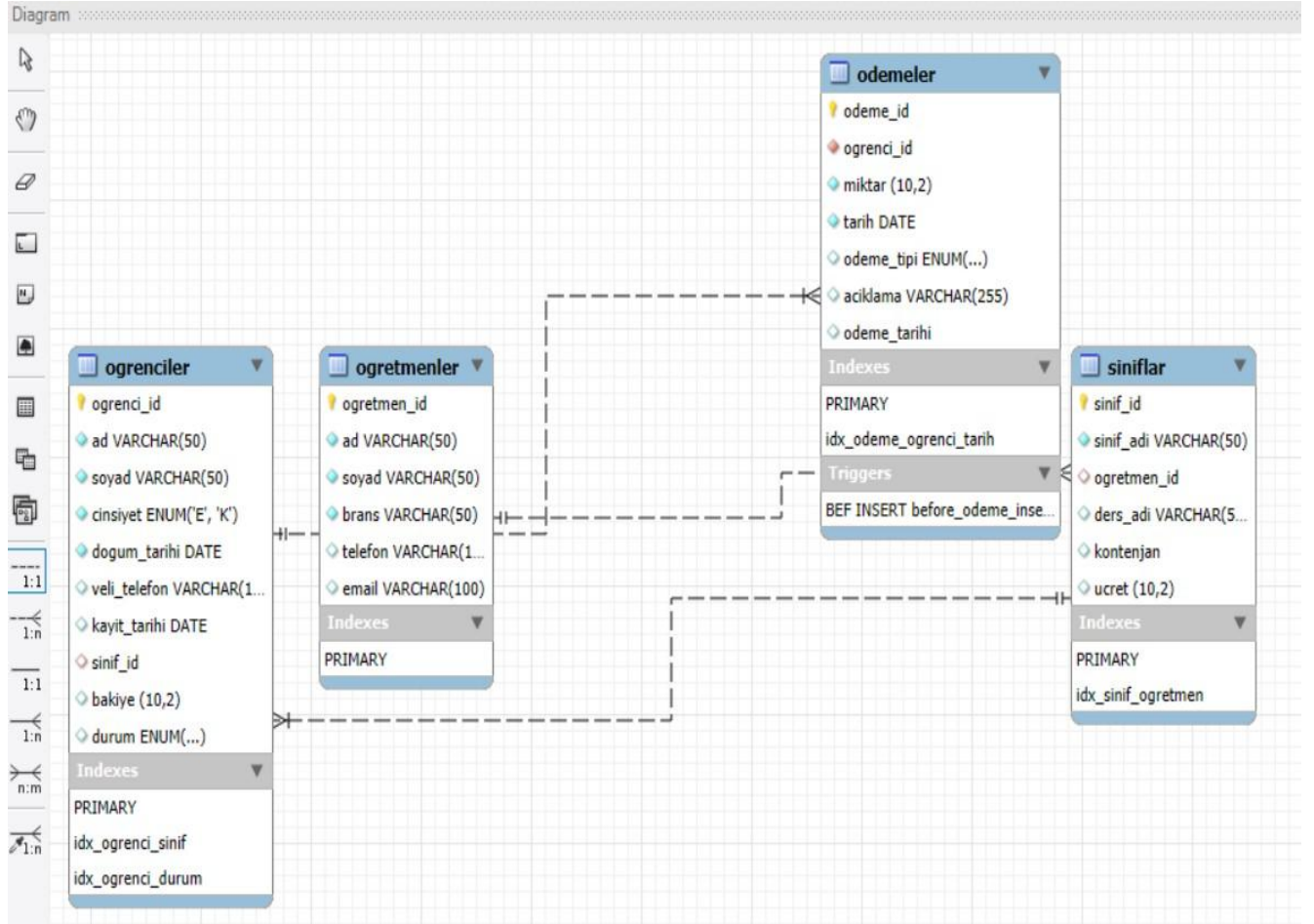
1.2 Sistem Özeti

Geliştirilen Dershane Öğrenci Takip Sistemi, dört ana bileşenden oluşmaktadır: öğretmen yönetimi, sınıf yönetimi, öğrenci yönetimi ve ödeme yönetimi. Sistemin en önemli özellikleri şunlardır:

- **Transaction yönetimi** ile ödeme kaydı ve bakiye güncellemesinin atomik olarak gerçekleştirilmesi
- **Trigger kullanımı** ile aynı gün içinde aynı öğrenci için ikinci ödemenin engellenmesi
- **Foreign key kısıtlamaları** ile referans bütünlüğünün sağlanması
- **İndeksleme** ile sorgu performansının optimize edilmesi

2. VERİTABANI TASARIMI

2.1 Varlık-İlişki Modeli



Diyagram Açıklaması:

1-Tablo Yapıları

- **Öğretmenler**: Öğretmen bilgilerini içerir
- **Sınıflar**: Sınıf ve ders bilgilerini, öğretmen ilişkisini tutar
- **Öğrenciler**: Öğrenci kişisel bilgileri, sınıf ilişkisi ve mali durumu
- **Ödemeler**: Tüm ödeme kayıtlarını saklar

2- İlişkiler:

- **Öğretmenler 7 Sınıflar (1:N)**: Bir öğretmen birden fazla sınıfa atanabilir

- **Sınıflar 7 Öğrenciler (1:N):** Bir sınıfta birden fazla öğrenci olabilir
- **Öğrenciler 7 Ödemeler (1:N):** Bir öğrencinin birden fazla ödemesi olabilir

3- Anahtar (Key) Yapısı:

- Her tablonun birincil anahtarı (primary key) mevcuttur
- Yabancı anahtarlar (foreign keys) tablolar arası ilişkiyi sağlar
- İndeksler (indexes) sorgu performansını optimize eder

2.2 Tablo Yapıları

Tablo 1: Öğretmenler Tablosu

Sütun Adı	Veri Tipi	Açıklama
ogretmen_id	INT	Birincil anahtar, otomatik artan
Ad	VARCHAR(50)	Öğretmen adı
soyad	VARCHAR(50)	Öğretmen soyadı
brans	VARCHAR(50)	Branş bilgisi
telefon	VARCHAR(15)	İletişim telefonu
email	VARCHAR(100)	E-posta adresi

	ogretmen_id	ad	soyad	brans	telefon	email
▶	1	Ahmet	Yılmaz	Matematik	05551234567	ahmet@dershane.com
	2	Ayşe	Demir	Fizik	05551234568	ayse@dershane.com
	3	Mehmet	Kaya	Kimya	05551234569	mehmet@dershane.com
	4	Fatma	Şahin	Biyoloji	05551234570	fatma@dershane.com
	5	Zeynep	Arslan	Geometri	05551234571	zeynep@dershane.com
✱	NULL	NULL	NULL	NULL	NULL	NULL

Tablo 2: Sınıflar Tablosu

Sütun Adı	Veri Tipi	Açıklama
sinif_id	INT	Birincil anahtar, otomatik artan
sinif_adi	VARCHAR(50)	Sınıf adı
Ogretmen_id	INT	Öğretmen referansı
Ders_adi	VARCHAR(50)	Ders adı
Kontenjan	INT	Maksimum öğrenci sayısı
Ucret	DECIMAL(10,2)	Sınıf ücreti

	sinif_id	sinif_adi	ogretmen_id	ders_adi	kontenjan	ucret
▶	1	Matematik-12A	1	Matematik	20	5000.00
	2	Fizik-12A	2	Fizik	15	4500.00
	3	Kimya-12B	3	Kimya	15	4500.00
	4	Biyoloji-12B	4	Biyoloji	15	4500.00
	5	Geometri-12C	5	Geometri	20	4000.00
	6	Matematik-11A	1	Matematik	20	4500.00
	7	Fizik-11A	2	Fizik	15	4000.00
•	NULL	NULL	NULL	NULL	NULL	NULL

Tablo 3: Öğrenciler Tablosu

Sütun Adı	Veri Tipi	Açıklama
Ogrenci_id	INT	Birincil anahtar, otomatik artan
ad	VARCHAR(50)	Öğrenci adı
soyad	VARCHAR(50)	Öğrenci soyadı
Cinsiyet	ENUM('E','K')	Cinsiyet bilgisi
Dogum_tarihi	DATE	Doğum tarihi
Veli_telefon	VARCHAR(15)	Veli iletişim numarası
Kayit_tarihi	DATE	Kayıt tarihi(otomatik)
Sinif_id	INT	Sınıf referansı
Bakiye	DECIMAL(10,2)	Toplam bakiye
durum	ENUM('Aktif','Pasif','Donuk')	Öğrenci durumu

[illegible]

3. UYGULAMA ÖZELLİKLERİ

3.1 TRANSACTION YÖNETİMİ

Ne Yapar? ⑦ Transaction (işlem), veri tabanında yapılan bir grup SQL komutunun atomik (bölünemez) bir şekilde çalışmasını sağlar. Ya tüm komutlar başarıyla çalışır, ya da hiçbiri çalışmaz.

Transaction kullanmazsak ne olur?

- Senaryo 1: Ödeme kaydedilir ama bakiye güncellenmez ⑦ Öğrenci borcu yanlış görünür
- Senaryo 2: Bakiye güncellenir ama ödeme kaydedilmez ⑦ Para alındı ama kaydı yok
- **Çözüm:** İkisini tek transaction içinde yapmak

Avantajları:

- Transaction sayesinde veri kaybı olmaz
- Hata durumunda otomatik geri alınır
- Tek satırla ödeme alınabilir: CALL OdemeAl(1, 1500, '2024-01-10', 'Nakit', 'İlk taksit');

START TRANSACTION; -- İşlem başlat

-- 1. ÖDEME KAYDET

INSERT INTO Odemeler (**ogrenci_id**, **miktar**, **tarih**, **odeme_tipi**, **aciklama**)

VALUES (**1**, **1500**, **'2024-01-10'**, **'Nakit'**, **'İlk taksit'**);

-- 2. BAKİYE GÜNCELLE

```
UPDATE Ogrenciler
SET bakiye = bakiye + 1500
WHERE ogrenci_id = 1;
COMMIT; -- kaydet
SELECT 'Ödeme başarıyla alındı' AS Mesaj;
END$$
```

Sorgunun Çıktısı:

114 20:15:08 INSERT INTO Odemeler (ogrenci_id, miktar, tarih, ödeme_tipi, aciklama) VALUES (1, 1500, '2024-01-10', 'Naki... Error Code: 1644. Aynı öğrenci için aynı gün ikinci ödeme yapılamaz

3.2 TRIGGER KULLANIMI

Trigger (tetikleyici), bir tabloda belirli bir olay (INSERT, UPDATE, DELETE) olduğunda otomatik çalışan SQL kodudur.

Çalışma Mantığı:

1. Yeni ödeme eklenirken tetiklenir
2. "Bu öğrenci bugün ödeme yaptı mı?" diye kontrol eder
3. Yaptıysa hata verir: 'Aynı öğrenci için aynı gün ikinci ödeme yapılamaz'

```
DELIMITER $$
```

```
CREATE TRIGGER before_odeme_insert
```

```
BEFORE INSERT ON Odemeler
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE v_count INT;
```

```
SET v_count = (  
    SELECT COUNT(*)  
    FROM Odemeler  
    WHERE ogrenci_id = NEW.ogrenci_id  
    AND DATE(tarih) = DATE(NEW.tarih)  
);  
IF v_count > 0 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Aynı öğrenci için aynı gün ikinci ödeme  
yapılamaz';  
END IF;  
END$$  
DELIMITER ;
```

Sorgunun çıktısı:

✓ 231 21:09:43 CREATE TRIGGER before_odeme_insert BEFORE ... 0 row(s) affected

3.3 İNDEKSLER

Ne Yapar? → Sorguları hızlandırır.

Neden İndeks Kullanıyoruz?

1. **Hız:** Sorgular daha hızlı çalışır
2. **Performans:** Büyük verilerde çok etkilidir
3. **Verimlilik:** CPU ve bellek kullanımını azaltır

Eklediğimiz İndeksler:

1. idx_ogrenci_sinif → Hangi öğrenci hangi sınıfta?

CREATE INDEX idx_ogrenci_sinif **ON** Ogrenciler(sinif_id);

2. idx_sinif_ogretmen → Hangi öğretmen hangi sınıfta?

CREATE INDEX idx_sinif_ogretmen **ON**
Siniflar(ogretmen_id);

3. idx_odeme_ogrenci_tarih → Öğrenci ödemelerini tarihe göre hızlı bul

CREATE INDEX idx_odeme_ogrenci_tarih **ON**
Odemeler(ogrenci_id, tarih);

4. idx_ogrenci_durum → Aktif/pasif öğrencileri hızlı liste

CREATE INDEX idx_ogrenci_durum **ON**
Ogrenciler(durum);

4. SORGULAR

4.1 İSTENEN SORGULAR

1. Her sınıftaki öğrenci sayısı

SELECT s.sinif_adi, **COUNT**(o.ogrenci_id) **as** ogrenci_sayisi
FROM Siniflar s
LEFT JOIN Ogrenciler o **ON** s.sinif_id = o.sinif_id
GROUP BY s.sinif_id;

Neden LEFT JOIN? 🕒 Öğrencisi olmayan sınıf da görünsün

Sorgunun çıktısı:

	sinif_adi	ogrenci_sayisi
▶	Matematik-12A	3
	Fizik-12A	2
	Kimya-12B	2
	Biyoloji-12B	0
	Geometri-12C	0
	Matematik-11A	2
	Fizik-11A	0

2. Belirli öğretmenin sınıfları

```
SELECT og.ad, og.soyad, s.sinif_adi, s.ders_adi
FROM Ogretmenler og
JOIN Siniflar s ON og.ogretmen_id = s.ogretmen_id
WHERE og.ogretmen_id = 1; ⑦ Ahmet Yılmaz
```

Sorgunun çıktısı:

	ad	soyad	sinif_adi	ders_adi
▶	Ahmet	Yılmaz	Matematik-12A	Matematik
	Ahmet	Yılmaz	Matematik-11A	Matematik

3. 5000 TL'den fazla ödeme yapanlar

```
SELECT o.ad, o.soyad, SUM(od.miktar) as toplam_odeme
FROM Ogrenciler o
JOIN Odemeler od ON o.ogrenci_id = od.ogrenci_id
GROUP BY o.ogrenci_id
HAVING toplam_odeme > 5000;
```

Sorgunun çıktısı:

	ad	soyad	toplam_odeme
▶	Ali	Çelik	7500.00
	Zeynep	Arslan	5500.00
	Ayşe	Demir	7000.00
	Seda	Toprak	6000.00

WHERE ve HAVING Farkı:

WHERE: Gruplamadan önce filtreler (doğum tarihine göre)

HAVING: Gruplamadan sonra filtreler (toplam ödemeye göre)

5. EKLER

EK A: Tüm SQL Kodları

-- 1. ÖĞRETMENLER TABLOSU

```
CREATE DATABASE IF NOT EXISTS derslane_yonetim;
```

```
USE derslane_yonetim;
```

```
CREATE TABLE Ogretmenler (
```

```
    ogretmen_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    ad VARCHAR(50) NOT NULL,
```

```
    soyad VARCHAR(50) NOT NULL,
```

```
    brans VARCHAR(50) NOT NULL,
```

```
    telefon VARCHAR(15),
```

```
    email VARCHAR(100)
```

```
);
```

-- 2. SINIFLAR TABLOSU

```
CREATE TABLE Siniflar (  
    sinif_id INT PRIMARY KEY AUTO_INCREMENT,  
    sinif_adi VARCHAR(50) NOT NULL,  
    ogretmen_id INT,  
    ders_adi VARCHAR(50),  
    kontenjan INT,  
    ucret DECIMAL(10,2)  
);
```

-- 3. ÖĞRENCİLER TABLOSU

```
CREATE TABLE Ogrenciler (  
    ogrenci_id INT PRIMARY KEY AUTO_INCREMENT,  
    ad VARCHAR(50) NOT NULL,  
    soyad VARCHAR(50) NOT NULL,  
    cinsiyet ENUM('E', 'K') NOT NULL,  
    dogum_tarihi DATE NOT NULL,  
    veli_telefon VARCHAR(15),  
    kayit_tarihi DATE DEFAULT (CURRENT_DATE),  
    sinif_id INT,  
    bakiye DECIMAL(10,2) DEFAULT 0.00,  
    durum ENUM('Aktif', 'Pasif', 'Donuk') DEFAULT 'Aktif'  
);
```

-- 4. ÖDEMELER TABLOSU

```
CREATE TABLE Odemeler (  
    odeme_id INT PRIMARY KEY AUTO_INCREMENT,  
    ogrenci_id INT NOT NULL,  
    miktar DECIMAL(10,2) NOT NULL,  
    tarih DATE NOT NULL,  
    odeme_tipi ENUM('Nakit', 'Kredi Kartı', 'Havale') DEFAULT  
'Nakit',  
    aciklama VARCHAR(255),  
    odeme_tarihi TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP  
);
```

-- FOREIGN KEY'LER

```
ALTER TABLE Siniflar  
ADD CONSTRAINT fk_sinif_ogretmen  
FOREIGN KEY (ogretmen_id) REFERENCES  
Ogretmenler(ogretmen_id)  
ON DELETE SET NULL;
```

```
ALTER TABLE Ogrenciler  
ADD CONSTRAINT fk_ogrenci_sinif  
FOREIGN KEY (sinif_id) REFERENCES Siniflar(sinif_id)  
ON DELETE SET NULL;
```



```
ALTER TABLE Odemeler
ADD CONSTRAINT fk_odeme_ogrenci
FOREIGN KEY (ogrenci_id) REFERENCES
Ogrenciler(ogrenci_id)
ON DELETE CASCADE;
```

```
-- İNDEKSLER
```

```
CREATE INDEX idx_ogrenci_sinif ON Ogrenciler(sinif_id);
CREATE INDEX idx_sinif_ogretmen ON Siniflar(ogretmen_id);
CREATE INDEX idx_odeme_ogrenci_tarih ON
Odemeler(ogrenci_id, tarih);
CREATE INDEX idx_ogrenci_durum ON Ogrenciler(durum);
```

```
-- TRIGGER
```

```
DELIMITER //
```

```
CREATE TRIGGER before_odeme_insert
BEFORE INSERT ON Odemeler
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
```

```
FROM Odemeler
WHERE ogrenci_id = NEW.ogrenci_id
AND DATE(tarih) = DATE(NEW.tarih)
) THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Aynı öğrenci için aynı gün ikinci
ödeme yapılamaz';

END IF;

END//
```

```
DELIMITER ;
```

```
-- STORED PROCEDURE
```

```
DELIMITER //
```

```
CREATE PROCEDURE OdemeAl(
    IN p_ogrenci_id INT,
    IN p_miktar DECIMAL(10,2),
    IN p_tarih DATE,
    IN p_odeme_tipi VARCHAR(20),
    IN p_aciklama VARCHAR(255)
)
BEGIN
```

DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

ROLLBACK;

RESIGNAL;

END;

START TRANSACTION;

INSERT INTO Odemeler (ogrenci_id, miktar, tarih,
odeme_tipi, aciklama)

VALUES (p_ogrenci_id, p_miktar, p_tarih, p_odeme_tipi,
p_aciklama);

UPDATE Ogrenciler

SET bakiye = bakiye + p_miktar

WHERE ogrenci_id = p_ogrenci_id;

COMMIT;

SELECT 'Ödeme başarıyla alındı' AS Mesaj;

END//

DELIMITER ;

-- 1. ÖĞRETMENLER

INSERT INTO Ogretmenler (ad, soyad, brans, telefon, email)
VALUES

('Ahmet', 'Yılmaz', 'Matematik', '05551234567',
'ahmet@dershane.com'),

('Ayşe', 'Demir', 'Fizik', '05551234568', 'ayse@dershane.com'),

('Mehmet', 'Kaya', 'Kimya', '05551234569',
'mehmet@dershane.com'),

('Fatma', 'Şahin', 'Biyoloji', '05551234570',
'fatma@dershane.com'),

('Zeynep', 'Arslan', 'Geometri', '05551234571',
'zeynep@dershane.com');

-- 2. SINIFLAR

INSERT INTO Siniflar (sinif_adi, ogretmen_id, ders_adi,
kontenjan, ucret) VALUES

('Matematik-12A', 1, 'Matematik', 20, 5000.00),

('Fizik-12A', 2, 'Fizik', 15, 4500.00),

('Kimya-12B', 3, 'Kimya', 15, 4500.00),

('Biyoloji-12B', 4, 'Biyoloji', 15, 4500.00),

('Geometri-12C', 5, 'Geometri', 20, 4000.00),

('Matematik-11A', 1, 'Matematik', 20, 4500.00),

('Fizik-11A', 2, 'Fizik', 15, 4000.00);

-- 3. ÖĞRENCİLER

INSERT INTO Ogrenciler (ad, soyad, cinsiyet, dogum_tarihi, veli_telefon, sinif_id, bakiye, durum) VALUES

-- Matematik-12A (5000+ ÖDEME YAPACAK ÖĞRENCİLER)

('Ali', 'Çelik', 'E', '2006-03-15', '05551234572', 1, 0, 'Aktif'), --
Toplam: 6500 TL

('Zeynep', 'Arslan', 'K', '2006-05-20', '05551234573', 1, 0, 'Aktif'),
-- Toplam: 5500 TL

('Mehmet', 'Yıldız', 'E', '2006-07-10', '05551234574', 1, 0, 'Aktif'),
-- Toplam: 3500 TL

-- Fizik-12A

('Ayşe', 'Demir', 'K', '2006-02-25', '05551234575', 2, 0, 'Aktif'),
-- Toplam: 7000 TL

('Can', 'Kara', 'E', '2006-08-30', '05551234576', 2, 0, 'Aktif'),
-- Toplam: 4500 TL

-- Kimya-12B

('Fatma', 'Şahin', 'K', '2006-01-12', '05551234577', 3, 0, 'Aktif'),
-- Toplam: 3000 TL

('Murat', 'Bulut', 'E', '2006-09-18', '05551234578', 3, 0, 'Pasif'),
-- Toplam: 0 TL

-- Matematik-11A

('Seda', 'Toprak', 'K', '2007-04-05', '05551234579', 6, 0, 'Aktif'),
-- Toplam: 6000 TL

('Burak', 'Deniz', 'E', '2007-06-22', '05551234580', 6, 0, 'Aktif');
-- Toplam: 2000 TL

-- ÖDEMELER

-- ALİ ÇELİK (ogrenci_id: 1) - TOPLAM: 6500 TL

CALL OdemeAl(1, 3000, '2024-01-10', 'Nakit', 'Kayıt ücreti');

CALL OdemeAl(1, 2000, '2024-02-10', 'Kredi Kartı', 'İkinci taksit');

CALL OdemeAl(1, 1500, '2024-03-10', 'Havale', 'Üçüncü taksit');

-- ZEYNEP ARSLAN (ogrenci_id: 2) - TOPLAM: 5500 TL

CALL OdemeAl(2, 2500, '2024-01-10', 'Nakit', 'Peşin ödeme');

CALL OdemeAl(2, 1500, '2024-02-10', 'Kredi Kartı', 'Ek ödeme');

CALL OdemeAl(2, 1500, '2024-03-10', 'Nakit', 'Tamamlama');

-- MEHMET YILDIZ (ogrenci_id: 3) - TOPLAM: 3500 TL

CALL OdemeAl(3, 2000, '2024-01-10', 'Nakit', 'İlk taksit');

CALL OdemeAl(3, 1500, '2024-02-10', 'Kredi Kartı', 'İkinci taksit');

-- AYŞE DEMİR (ogrenci_id: 4) - TOPLAM: 7000 TL

CALL OdemeAl(4, 4000, '2024-01-05', 'Nakit', 'Peşin ödeme - kampanya');

CALL OdemeAl(4, 3000, '2024-02-05', 'Havale', 'Ek kayıt');

-- CAN KARA (ogrenci_id: 5) - TOPLAM: 4500 TL

CALL OdemeAl(5, 4500, '2024-01-05', 'Nakit', 'Peşin ödeme');

-- FATMA ŞAHİN (ogrenci_id: 6) - TOPLAM: 3000 TL

CALL OdemeAl(6, 1500, '2024-01-08', 'Nakit', 'İlk taksit');

CALL OdemeAl(6, 1500, '2024-02-08', 'Kredi Kartı', 'İkinci taksit');

-- SEDA TOPRAK (ogrenci_id: 8) - TOPLAM: 6000 TL

CALL OdemeAl(8, 3000, '2024-01-12', 'Havale', 'Peşin ödeme');

CALL OdemeAl(8, 3000, '2024-02-12', 'Kredi Kartı', 'Ek ders ücreti');

-- BURAK DENİZ (ogrenci_id: 9) - TOPLAM: 2000 TL

CALL OdemeAl(9, 2000, '2024-01-12', 'Nakit', 'Peşin ödeme');

-- SORGULAR

-- SORGU 1: Her sınıftaki öğrenci sayısını listele

```
SELECT s.sinif_adi, COUNT(o.ogrenci_id) as ogrenci_sayisi  
FROM Siniflar s  
LEFT JOIN Ogrenciler o ON s.sinif_id = o.sinif_id  
GROUP BY s.sinif_id;
```

-- SORGU 2: Belirli bir öğretmenin verdiği sınıfları göster

```
SELECT og.ad, og.soyad, s.sinif_adi, s.ders_adi  
FROM Ogretmenler og  
JOIN Siniflar s ON og.ogretmen_id = s.ogretmen_id  
WHERE og.ogretmen_id = 1; -- Ahmet Yılmaz
```

-- SORGU 3: 5000tl den fazla ödeme yapanlar

```
SELECT o.ad, o.soyad, SUM(od.miktar) as toplam_odeme  
FROM Ogrenciler o  
JOIN Odemeler od ON o.ogrenci_id = od.ogrenci_id  
GROUP BY o.ogrenci_id  
HAVING toplam_odeme > 5000;
```

-- TRIGGER TESTİ

-- Test 1: Normal ödeme

```
CALL OdemeAI(1, 1000, CURDATE(), 'Nakit', 'Trigger test - ilk  
ödeme');
```


-- Test 2: Aynı gün ikinci ödeme (HATA VERMELİ!)

```
CALL OdemeAl(1, 500, CURDATE(), 'Kredi Kartı', 'Trigger test -  
ikinci ödeme');
```

-- HATA: "Aynı öğrenci için aynı gün ikinci ödeme yapılamaz"

-- Test 3: Farklı gün ödeme (Başarılı)

```
CALL OdemeAl(1, 800, DATE_ADD(CURDATE(), INTERVAL 1  
DAY), 'Havale', 'Trigger test - farklı gün');
```

-- Öğrenci bakiyeleri

```
SELECT
```

```
    ogrenci_id,
```

```
    CONCAT(ad, ' ', soyad) AS ogrenci_adi,
```

```
    bakiye,
```

```
    durum
```

```
FROM Ogrenciler
```

```
ORDER BY bakiye DESC;
```

-- Ödeme kayıtları

```
SELECT
```

```
    odeme_id,
```

ogrenci_id,

miktar,

tarih,

odeme_tipi,

aciklama

FROM Odemeler

ORDER BY tarih DESC, odeme_id DESC

LIMIT 10;